

32. Construct a C program to simulate the Least Recently Used paging technique of memory management.

AIM :

Construct a C program to simulate the Least Recently Used paging technique of memory management.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX_FRAMES 3
```

```
void printFrames(int frames[], int n)
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        if (frames[i] == -1)
```

```
        {
```

```
            printf(" - ");
```

```
        }
```

```
        else
```

```
        {
```

```
            printf(" %d ", frames[i]);
```

```
        }
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main()
```

```
{
```

```
    int frames[MAX_FRAMES];
```

```
    int usageHistory[MAX_FRAMES];
```

```
    int pageFaults = 0;
```

```
    int i, j, k;
```

```
    int referenceString[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
```

```
    int n = sizeof(referenceString) / sizeof(referenceString[0]);
```

```
    /* Initialize frames */
```

```
    for (i = 0; i < MAX_FRAMES; i++)
```

```
    {
```

```
        frames[i] = -1;
```

```
        usageHistory[i] = 0;
```

```
    }
```

```
    /* Print reference string */
```

```
    printf("Reference String: ");
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        printf("%d ", referenceString[i]);
```

```
    }
```

```
printf("\n\n");
printf("Page Replacement Order:\n");

/* LRU Page Replacement */
for (i = 0; i < n; i++)
{
    int page = referenceString[i];
    int pageFound = 0;

    /* Check whether page is already in memory */
    for (j = 0; j < MAX_FRAMES; j++)
    {
        if (frames[j] == page)
        {
            pageFound = 1;

            /* Increase usage history of other pages */
            for (k = 0; k < MAX_FRAMES; k++)
            {
                if (k != j && frames[k] != -1)
                {
                    usageHistory[k]++;
                }
            }

            /* Reset usage counter of used page */
            usageHistory[j] = 0;
        }
    }
}
```

```

        break;
    }
}

/* Page fault */
if (!pageFound)
{
    int lruPage = 0;
    int replacedPage;

    printf("Page %d -> ", page);

    /* Find an empty frame first */
    for (j = 0; j < MAX_FRAMES; j++)
    {
        if (frames[j] == -1)
        {
            lruPage = j;
            break;
        }
    }

    /* If no empty frame, find LRU page */
    if (frames[lruPage] != -1)
    {
        for (j = 1; j < MAX_FRAMES; j++)

```

```

    {
        if (usageHistory[j] > usageHistory[lruPage])
        {
            lruPage = j;
        }
    }
}

replacedPage = frames[lruPage];

frames[lruPage] = page;
usageHistory[lruPage] = 0;

if (replacedPage != -1)
{
    printf("Replace %d with %d: ",
           replacedPage, page);
}
else
{
    printf("Load into an empty frame: ");
}

printFrames(frames, MAX_FRAMES);

pageFaults++;
}

```

```
}
```

```
printf("\nTotal Page Faults: %d\n", pageFaults);
```

```
return 0;
```

```
}
```

OUTPUT:

```
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
```

```
Page Replacement Order:
```

```
Page 7 -> Load into an empty frame: 7 - -
```

```
Page 0 -> Load into an empty frame: 7 0 -
```

```
Page 1 -> Load into an empty frame: 7 0 1
```

```
Page 2 -> Replace 7 with 2: 2 0 1
```

```
Page 3 -> Replace 2 with 3: 3 0 1
```

```
Page 4 -> Replace 1 with 4: 3 0 4
```

```
Page 2 -> Replace 3 with 2: 2 0 4
```

```
Page 3 -> Replace 2 with 3: 3 0 4
```

```
Page 2 -> Replace 4 with 2: 3 0 2
```

```
Total Page Faults: 9
```

```
...Program finished with exit code 0
```

```
Press ENTER to exit console.
```